Qualité logicielle et tests avancés

Informatique — Licence 3 (L3)

Cours INF-L3-15

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Garantir la qualité d'un logiciel de façon professionnelle : modèles de qualité, pyramide des tests, tests unitaires automatisés avec pytest, tests d'API et de bout en bout, couverture de code branchée sur les pipelines, gestion rigoureuse des anomalies et campagnes de test pilotées par un plan.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Concepts de qualité et modèles d'assurance qualité
• Leçon 2 — Stratégies de test : la pyramide des tests
• Leçon 3 — Tests unitaires automatisés avec un framework de test
• Leçon 4 — Tests d'API et tests de bout en bout
• Leçon 5 — Couverture de code et intégration aux pipelines
• Leçon 6 — Gestion des anomalies et campagnes de test
• Leçon 7 — Projet : plan de test complet d'une application réelle
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Un logiciel « qui marche » sur la machine de son auteur n'est pas encore un logiciel de qualité. Entre le premier prototype et le produit que des centaines d'utilisateurs emploient chaque jour, il y a un travail méthodique : définir ce que « qualité » veut dire pour ce produit, vérifier systématiquement que le code fait ce qu'il doit, et prouver ces vérifications par des tests que l'on peut relancer à volonté. C'est ce travail que ce cours t'apprend à organiser et à outiller.

Nous travaillerons sur un fil rouge unique : BiblioIAM, l'application de gestion de la bibliothèque de l'institut. Chaque leçon y ajoute une couche de protection réelle : une grille d'audit chiffrée, une pyramide de tests chiffrée elle aussi, des suites pytest exécutées pour de vrai — tu pourras relancer exactement les mêmes commandes —, un rapport de couverture interprété sans complaisance, une fiche d'anomalie complète et, pour finir, un plan de test de niveau professionnel.

Le cours prolonge le génie logiciel de L2 et le DevOps de L3 : les tests que tu écriras ici sont ceux que les pipelines du cours INF-L3-12 exécutent à chaque intégration. À la fin, tu sauras non seulement écrire des tests, mais surtout décider quoi tester, à quel niveau, avec quels critères d'arrêt — c'est-à-dire passer du réflexe « je clique pour voir » à une véritable démarche qualité d'ingénieur.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Définir la qualité logicielle et la mesurer avec les caractéristiques du modèle ISO/IEC 25010.
- Distinguer assurance qualité, contrôle qualité et test, et situer chacun dans le cycle de vie.
- Construire une stratégie de test équilibrée selon la pyramide des tests.
- Écrire et exécuter des tests unitaires automatisés avec pytest, en couvrant classes d'équivalence et valeurs limites.
- Tester une API REST et décrire un test de bout en bout sur un parcours utilisateur complet.
- Mesurer une couverture de code, interpréter son rapport sans le surinterpréter, et l'intégrer à un pipeline d'intégration continue.
- Rédiger une fiche d'anomalie complète, classer gravité et priorité, et piloter une campagne de test jusqu'à la décision de recette.

Leçon 1 — Concepts de qualité et modèles d'assurance qualité

La qualité logicielle est l'aptitude d'un produit à satisfaire des besoins explicites et implicites, dans des conditions d'utilisation données. Elle ne se réduit pas à « sans bogue » : un logiciel sans aucun défaut connu mais inutilisable, lent ou impossible à faire évoluer est un produit de mauvaise qualité. Pour raisonner de façon structurée, le modèle ISO/IEC 25010 découpe la qualité du produit en huit caractéristiques : adéquation fonctionnelle, efficacité de performance, compatibilité, utilisabilité, fiabilité, sécurité, maintenabilité et portabilité. Chaque caractéristique se décline en questions mesurables : le produit fait-il tout ce qui est demandé ? répond-il vite ? tombe-t-il rarement en panne ?

Il faut ensuite distinguer trois activités souvent confondues. L'assurance qualité organise les processus pour prévenir les défauts : normes de codage, revues, listes de contrôle, définition de « terminé ». Le contrôle qualité vérifie le produit obtenu : inspections et relectures de documents ou de code, sans nécessairement exécuter le programme. Le test, lui, exécute le logiciel avec des données choisies pour révéler des écarts entre le comportement observé et le comportement attendu. Assurance, contrôle et test sont complémentaires : tester un produit mal conçu dès le départ revient à mesurer très précisément un échec organisé.

Un piège classique consiste à croire que le test peut démontrer l'absence de défauts. C'est impossible dès que les entrées sont nombreuses : on ne peut pas tout essayer. Le test démontre la présence de défauts, jamais leur absence — c'est un des principes fondateurs du syllabus ISTQB. La démarche professionnelle consiste donc à concentrer l'effort là où le risque est le plus élevé : fonctions critiques pour les utilisateurs, zones du code souvent modifiées, calculs d'argent ou de sécurité. C'est ce raisonnement par les risques qui guide la stratégie des leçons suivantes.

L'essentiel
La qualité se définit et se mesure (modèle ISO/IEC 25010, 8 caractéristiques). L'assurance qualité prévient, le contrôle qualité inspecte, le test exécute. Et le test montre la présence de défauts, jamais leur absence : on concentre donc l'effort sur les risques.

Exemple — Grille d'audit qualité pondérée de BiblioIAM

```
Grille d'audit qualité — BiblioIAM v1.2 (notes de 0 à 5)
Caractéristique (ISO/IEC 25010) Note Poids Contribution
Adéquation fonctionnelle 4 25 % 1,00
Fiabilité 3 20 % 0,60
Utilisabilité 4 15 % 0,60
Sécurité 2 20 % 0,40
Maintenabilité 3 10 % 0,30
Efficacité de performance 4 10 % 0,40
-----------------------------------------------------------
Score pondéré : 3,30 / 5 (66 %)
Verdict : ensemble correct, mais la sécurité (2/5) est le
chantier prioritaire avant l'ouverture au public.
```

À retenir

- La qualité couvre bien plus que l'absence de bogues : utilisabilité, fiabilité, sécurité, maintenabilité…
- Assurance qualité = prévenir par les processus ; contrôle qualité = inspecter ; test = exécuter et observer.
- Comme on ne peut pas tout tester, on priorise par le risque : critique, souvent modifié, sensible.

Exercice

Une application de gestion de cantine scolaire reçoit l'audit suivant (poids égaux de 25 % sur quatre caractéristiques) : adéquation fonctionnelle 5/5, fiabilité 2/5 (elle plante environ une fois par semaine à l'heure de pointe), sécurité 3/5, utilisabilité 4/5. Calcule le score pondéré, puis désigne LA caractéristique à traiter en premier en justifiant ton choix par le risque, pas seulement par la note.

✔ Corrigé
Poids égaux : le score pondéré est la moyenne simple : (5 + 2 + 3 + 4) ÷ 4 = 14 ÷ 4 = 3,5/5, soit 70 %. La caractéristique à traiter en premier est la fiabilité : un plantage hebdomadaire à l'heure de pointe bloque le service des repas, donc touche tous les utilisateurs au pire moment — le risque (probabilité élevée × impact fort) dépasse celui de la sécurité notée 3, dont aucune faille concrète n'est signalée. La leçon : une grille sert à décider, pas seulement à noter ; à score égal, on traite d'abord ce qui peut faire échouer le service réel.

Leçon 2 — Stratégies de test : la pyramide des tests

Tous les tests ne coûtent pas le même prix et ne donnent pas le même retour. Un test unitaire s'exécute en quelques millisecondes, isole une fonction et désigne précisément la ligne fautive quand il échoue. Un test de bout en bout, qui pilote l'application complète comme un utilisateur, prend des secondes ou des minutes, casse pour des raisons parfois éloignées du défaut réel (un sélecteur d'écran modifié suffit) et demande une maintenance constante. La pyramide des tests, popularisée par Mike Cohn, tire la conséquence de ce constat : beaucoup de tests unitaires rapides à la base, un nombre moyen de tests d'intégration et d'API au milieu, très peu de tests de bout en bout au sommet, réservés aux parcours critiques.

L'anti-modèle est tout aussi célèbre : le « cornet de glace » (ice-cream cone), où l'équipe ne jure que par les tests manuels et les tests d'interface. La suite devient lente, fragile et redoutée ; on finit par ne plus la lancer, exactement au moment où le produit en a le plus besoin. La bonne répartition n'est pas un dogme chiffré, mais un équilibre économique : chaque niveau doit apporter une information que les niveaux inférieurs ne peuvent pas donner. Si un comportement peut être vérifié par un test unitaire, il est absurde de le vérifier uniquement à travers l'interface graphique.

Une stratégie de test complète précise, pour chaque niveau : ce qui est testé, par qui, avec quels outils, à quel moment (à chaque commit, à chaque fusion, chaque nuit, avant livraison) et avec quels critères pour décider que « ça suffit ». Elle inclut aussi ce qui ne sera PAS automatisé — certains contrôles visuels ou exploratoires restent manuels, et le dire noir sur blanc fait partie de la stratégie. Les deux leçons suivantes mettent en œuvre les deux étages du bas et du milieu de la pyramide sur BiblioIAM.

L'essentiel
Pyramide des tests : à la base, de nombreux tests unitaires rapides et précis ; au milieu, des tests d'intégration et d'API ; au sommet, quelques tests de bout en bout sur les parcours critiques. L'inverse (le « cornet de glace ») donne des suites lentes et fragiles que plus personne ne lance.

Exemple — Pyramide des tests et répartition chiffrée du plan BiblioIAM

```
La pyramide des tests (répartition conseillée)
/\
/ \ Tests de bout en bout (E2E)
/----\ parcours complets, lents : PEU
/ \ Tests d'intégration et d'API
/--------\ échanges entre composants : MOYEN
/ \ Tests unitaires
/------------\ fonctions isolées, rapides : BEAUCOUP

Extrait du plan de test de BiblioIAM v1.2
- 186 tests unitaires ......... environ 40 s (à chaque commit)
- 34 tests d'API/intégration ... environ 3 min (à chaque fusion)
- 9 tests de bout en bout ...... environ 12 min (chaque nuit)
Règle : un échec à un étage bloque la promotion vers l'étage
suivant ; on ne « répare » pas un E2E en supprimant le test.
```

À retenir

- Plus un test est haut dans la pyramide, plus il est lent, coûteux à maintenir et imprécis sur la cause d'échec.
- Automatiser au plus bas niveau capable de vérifier le comportement visé.
- Une stratégie dit aussi ce qui ne sera pas automatisé, et à quel moment chaque étage est exécuté.

Exercice

Une équipe présente sa répartition : 20 tests unitaires, 45 tests d'intégration, 60 tests de bout en bout, et se plaint que « la suite complète prend plus de deux heures, alors on ne la lance qu'avant les livraisons ». 1) Nomme l'anti-modèle. 2) Vérifie le grief : si chaque test de bout en bout dure en moyenne 2 minutes, combien de temps prennent-ils à eux seuls ? 3) Propose deux rééquilibrages concrets.

✔ Corrigé
1) C'est le « cornet de glace » : la pyramide est inversée, le niveau le plus lent et fragile est le plus fourni. 2) 60 × 2 min = 120 minutes = 2 heures pour les seuls tests de bout en bout : le grief est fondé, et il est structurel, pas conjoncturel. 3) Rééquilibrages : descendre au niveau unitaire tout ce qui teste des règles de calcul ou de validation (la majorité des 60 scénarios vérifient en réalité des règles métier accessibles sans interface) ; ne conserver en bout en bout que les parcours critiques de bout en bout (connexion, emprunt complet, retour) et transformer les autres scénarios en tests d'API, dix à cinquante fois plus rapides. Objectif : une suite complète en quelques minutes, relancée à chaque fusion.

Leçon 3 — Tests unitaires automatisés avec un framework de test

Un test unitaire vérifie une unité de code — le plus souvent une fonction — de façon isolée, déterministe et automatique. En Python, deux frameworks dominent : unittest, fourni avec le langage, et pytest, devenu le standard de fait pour sa syntaxe minimale : une fonction dont le nom commence par test_, des assert ordinaires, et pytest

découvre et exécute tout seul. Un bon test suit le schéma AAA : Arrange (préparer les données), Act (appeler la fonction), Assert (vérifier le résultat). Il ne dépend ni de l'heure, ni du réseau, ni de l'ordre de passage des autres tests, sinon il devient « instable » et l'équipe cesse de croire ses verdicts.

Écrire un test par fonction « en moyenne » ne suffit pas : il faut viser les endroits où les défauts naissent. Deux techniques de boîte noire guident le choix des cas. Le partitionnement en classes d'équivalence regroupe les entrées qui doivent recevoir le même traitement : pour un tarif de retard, « aucun retard », « première semaine » et « au-delà » sont trois classes — un représentant par classe suffit souvent. L'analyse des valeurs limites ajoute les bornes exactes, car c'est là que les erreurs de comparaison (< au lieu de ≤) se cachent : on teste la borne elle-même et ses voisines immédiates. L'exemple qui suit applique les deux techniques à la fonction de calcul des frais de retard de BiblioIAM.

La suite ci-dessous a été réellement exécutée avec pytest : les sept cas passent. Remarque le septième test : il vérifie qu'une entrée invalide (un nombre de jours négatif) déclenche bien l'exception prévue, grâce à pytest.raises. Un test qui ne peut pas échouer ne sert à rien ; avant de faire confiance à une suite, un bon réflexe est de dégrader volontairement le code (changer 100 en 150) et de vérifier que les tests deviennent rouges — c'est le principe du test de mutation artisanal, et c'est ce que fera le TP de ce cours sur un module volontairement fautif.

L'essentiel
pytest : des fonctions test_, des assert, une découverte automatique. On choisit les cas par classes d'équivalence et valeurs limites, on teste aussi les erreurs attendues (pytest.raises), et on vérifie qu'un test peut réellement échouer avant de lui faire confiance.

Exemple — Tester frais_retard par classes d'équivalence et valeurs limites

```
# biblio.py (extrait) — frais de retard d'un emprunt, en F CFA
def frais_retard(jours):
if jours < 0:
raise ValueError("jours négatif impossible")
if jours == 0:
return 0
if jours <= 7:
return 100 * jours
frais = 700 + 200 * (jours - 7)
return min(frais, 3000)

# test_biblio.py — un cas par classe, toutes les bornes
import pytest
from biblio import frais_retard

def test_sans_retard(): assert frais_retard(0) == 0
def test_premier_jour(): assert frais_retard(1) == 100
def test_dans_la_semaine(): assert frais_retard(5) == 500
def test_borne_semaine(): assert frais_retard(7) == 700
def test_au_dela(): assert frais_retard(10) == 1300
def test_plafond(): assert frais_retard(30) == 3000
def test_negatif_refuse():
with pytest.raises(ValueError):
frais_retard(-1)

# Exécution réelle (pytest 9.1.1) : 7 passed in 0.02s
```

À retenir

• Un test unitaire est isolé, déterministe et rapide ; s'il dépend du réseau ou de l'horloge, ce n'est plus un test unitaire.

• Classes d'équivalence pour couvrir les comportements, valeurs limites pour traquer les erreurs de comparaison.

• Tester aussi les cas d'erreur : une exception prévue et vérifiée vaut mieux qu'un plantage découvert en production.

Exercice

Le module biblio.py contient aussi : def peut_emprunter(nb_emprunts, bloque) qui renvoie False si le membre est bloqué, sinon True tant qu'il a moins de 3 emprunts en cours. Écris la suite de tests de cette fonction en appliquant les valeurs limites et la priorité du blocage. Combien de cas sont nécessaires au minimum, et pourquoi tester (0, True) ET (5, True) ?

✔ Corrigé
Quatre cas suffisent et couvrent tout : peut_emprunter(2, False) vaut True (sous la limite) ; peut_emprunter(3, False) vaut False (la borne exacte : 3 emprunts atteints, refus) ; peut_emprunter(0, True) vaut False (le blocage refuse même sans aucun emprunt) ; peut_emprunter(5, True) vaut False (le blocage refuse aussi au-delà de la limite). Tester (0, True) et (5, True) n'est pas un luxe : le premier prouve que le blocage seul suffit à refuser, le second que le code ne court-circuite pas le test de blocage quand la limite est déjà dépassée — deux chemins de décision différents. Exécutée avec pytest, cette suite donne : 4 passed. Elle servira à nouveau dans la leçon 5 pour compléter la couverture du module.

Leçon 4 — Tests d'API et tests de bout en bout

Les tests d'API vérifient les échanges entre composants : on envoie de vraies requêtes HTTP à un service et on contrôle le code de statut, la structure de la réponse et son contenu métier. Ils attrapent une famille de défauts invisibles aux tests unitaires : routes mal nommées, codes de statut incohérents (un 200 là où un 201 est promis), champs manquants, validations absentes côté serveur. Ils restent rapides — quelques dizaines de millisecondes par requête sur un serveur de test local — et beaucoup plus stables que les tests d'interface, car le contrat d'une API change bien moins souvent que ses écrans.

Pour les rendre reproductibles, on lance l'API sur un environnement de test maîtrisé : serveur local démarré par une fixture, base de données de test réinitialisée, données connues à l'avance. Dans l'exemple exécuté ci-dessous, la fixture base_url (définie dans conftest.py) démarre un mini-serveur BiblioIAM sur un port libre ; les quatre tests vérifient un succès (200 sur la liste des livres), une ressource absente (404), une requête invalide rejetée (400 quand le champ membre manque) et une création (201 pour un emprunt complet). Les quatre passent réellement.

Au sommet de la pyramide, les tests de bout en bout (E2E) déroulent un parcours complet comme le ferait un utilisateur : se connecter, chercher un livre, l'emprunter, vérifier le reçu. Ils apportent une confiance unique — tout le système assemblé fonctionne — mais ils sont lents, sensibles aux changements d'interface et difficiles à diagnostiquer quand ils échouent. On les réserve donc aux quelques parcours dont la panne serait catastrophique, on les exécute typiquement la nuit ou avant livraison, et on accepte leur coût comme une assurance, pas comme un outil de développement quotidien.

L'essentiel
Tests d'API : de vraies requêtes contre un service de test, en vérifiant statut HTTP et contenu — rapides et stables, ils attrapent les défauts de contrat. Tests de bout en bout : des parcours utilisateur complets, précieux mais lents et fragiles, donc rares et réservés aux parcours critiques.

Exemple — Quatre tests d'API exécutés contre un serveur de test local

```
# test_api.py — API BiblioIAM lancée sur un serveur local
# (fixture base_url définie dans conftest.py : voir le texte)
import json, urllib.error, urllib.request
import pytest

def test_liste_des_livres(base_url):
with urllib.request.urlopen(base_url + "/livres") as rep:
assert rep.status == 200
livres = json.loads(rep.read())
assert livres[0]["titre"] == "Algorithmique"

def test_livre_introuvable(base_url):
with pytest.raises(urllib.error.HTTPError) as info:
urllib.request.urlopen(base_url + "/livres/999")
assert info.value.code == 404

def test_emprunt_sans_membre_refuse(base_url):
req = urllib.request.Request(base_url + "/emprunts",
data=b"{}", method="POST",
headers={"Content-Type": "application/json"})
with pytest.raises(urllib.error.HTTPError) as info:
urllib.request.urlopen(req)
assert info.value.code == 400

def test_emprunt_valide_cree(base_url):
corps = json.dumps({"membre": 12, "livre": 1}).encode()
req = urllib.request.Request(base_url + "/emprunts",
data=corps, method="POST",
headers={"Content-Type": "application/json"})
with urllib.request.urlopen(req) as rep:
assert rep.status == 201

# Exécution réelle contre le serveur de test : 4 passed
```

À retenir

• Un test d'API vérifie le contrat : code de statut, structure et contenu — pas seulement « ça répond ».

• Environnement de test maîtrisé (fixture, données connues) = tests reproductibles, condition de leur crédibilité.

• Les E2E couvrent peu de parcours, mais les plus critiques ; leur lenteur est un coût assumé, pas un défaut à nier.

Exercice

Ajoute à la suite d'API un test du détail d'un livre existant : GET /livres/1 doit répondre 200, avec le titre « Algorithmique » et le champ disponible à vrai. Écris le test avec les mêmes outils que l'exemple (urllib et pytest), en expliquant pourquoi vérifier le contenu, et pas seulement le code 200, rend le test plus fort.

✔ Corrigé
def test_detail_dun_livre(base_url): on ouvre base_url + "/livres/1", on assert rep.status == 200, on décode le JSON, puis on assert livre["titre"] == "Algorithmique" et livre["disponible"] is True. Ce test passe réellement contre le serveur de test. Vérifier le contenu est décisif : un code 200 peut accompagner une réponse fausse (mauvais livre, champ renommé, disponibilité inversée) — c'est exactement la famille de défauts que les tests d'API existent pour attraper. Un test qui ne contrôle que le statut ressemble à un contrôle qualité qui vérifierait qu'une lettre est arrivée sans jamais l'ouvrir.

Leçon 5 — Couverture de code et intégration aux pipelines

La couverture de code mesure quelle proportion du code source a été exécutée au moins une fois pendant les tests. L'outil de référence en Python, coverage.py, produit un rapport par fichier avec, pour chacun, le nombre d'instructions, le nombre d'instructions manquées, le pourcentage couvert et — c'est le plus utile — la liste des lignes jamais exécutées. La couverture par ligne est la plus courante ; la couverture de branches, plus fine, vérifie que chaque alternative d'un if a été prise dans les deux sens. Ces chiffres transforment une impression (« on a pas mal de tests ») en fait observable et discutable en équipe.

Le rapport réel ci-dessous, obtenu en lançant les tests unitaires de la leçon 3 sur le module complet biblio.py (trois fonctions), est un cas d'école : 58 % au total, mais ce chiffre cache l'essentiel. frais_retard est couvert à 100 % ; en revanche les lignes 16-18 (peut_emprunter) et 23-27 (message_retard) n'ont jamais été exécutées : deux fonctions entières sans aucun test. En ajoutant les tests de l'exercice de la leçon 3 et trois cas pour message_retard (0 jour, 1 jour, plusieurs jours), la même commande donne 14 tests réussis et 100 % — nous l'avons vérifié par l'exécution. Lis toujours la colonne Missing avant le pourcentage.

Reste la limite fondamentale, à ne jamais oublier : la couverture mesure l'exécution, pas la vérification. Un test qui appelle une fonction sans aucun assert « couvre » ses lignes et ne prouve rien. Un module couvert à 100 % peut donc contenir des défauts — constantes fausses jamais confrontées à une valeur attendue, cas aux limites non envisagés. Le bon usage est donc : un seuil plancher dans le pipeline (par exemple, le pipeline échoue si la couverture du code métier passe sous 80 %), la lecture des lignes manquantes à chaque revue, et jamais d'objectif de 100 % à tout prix, qui pousse à écrire des tests vides pour faire monter le compteur. Intégrée ainsi à la CI du cours INF-L3-12, la couverture devient un filet, pas un trophée.

L'essentiel
La couverture dit quelles lignes ont été exécutées par les tests — la colonne des lignes manquantes vaut plus que le pourcentage. Elle ne dit pas si les tests vérifient les bons résultats : 100 % de couverture n'a jamais signifié zéro bogue. On l'utilise comme seuil plancher dans le pipeline, pas comme objectif absolu.

Exemple — Rapport de couverture réel de biblio.py et sa lecture

```
# On mesure la couverture des tests unitaires de la leçon 3
$ coverage run -m pytest test_biblio.py
$ coverage report -m --omit="test_*"

Name Stmts Miss Cover Missing
----------------------------------------
biblio.py 19 8 58% 16-18, 23-27
----------------------------------------
TOTAL 19 8 58%

# Lecture : les lignes 16-18 (peut_emprunter) et 23-27
# (message_retard) n'ont jamais été exécutées : ces deux
# fonctions n'ont AUCUN test. frais_retard, lui, est
# couvert à 100 %. Le chiffre global (58 %) ne dit ni
# où sont les trous, ni si les tests vérifient les bons
# résultats : c'est la colonne Missing qui guide l'action.
```

À retenir

• Lire un rapport de couverture : d'abord les lignes manquantes, ensuite seulement le pourcentage.

• Couverture = exécution mesurée, pas correction prouvée ; un test sans assert couvre sans vérifier.

• Dans un pipeline : un seuil plancher qui bloque la fusion, pas une course au 100 %.

Exercice

À partir du rapport de la leçon, écris les tests manquants pour que biblio.py atteigne 100 % de couverture d'instructions : il faut couvrir peut_emprunter (déjà fait dans l'exercice de la leçon 3) et message_retard, qui renvoie « Aucun retard. » pour 0, « 1 jour de retard. » pour 1 et « N jours de retard. » sinon. Combien de cas pour message_retard, et quel résultat complet attends-tu (nombre de tests, couverture) ?

✔ Corrigé

Trois cas pour message_retard : message_retard(0) == « Aucun retard. », message_retard(1) == « 1 jour de retard. », message_retard(5) == « 5 jours de retard. » — un par branche de la fonction, y compris la branche du singulier, facile à oublier. En réunissant les trois fichiers (7 tests de frais_retard, 4 de peut_emprunter, 3 de message_retard), l'exécution réelle donne : 14 passed, et le rapport de couverture affiche biblio.py : 19 instructions, 0 manquée, 100 %. Attention à la conclusion honnête : ce 100 % garantit que chaque ligne a été exécutée et confrontée aux valeurs attendues choisies — il ne garantit pas que les règles métier elles-mêmes sont les bonnes.

Leçon 6 — Gestion des anomalies et campagnes de test

Une anomalie (ou défaut, ou bogue) ne devient gérable que lorsqu'elle est décrite de façon complète et reproductible. Une fiche d'anomalie professionnelle contient : un identifiant unique, un titre clair, le projet et la version, l'environnement (système, navigateur, données utilisées), les étapes de reproduction numérotées, le résultat observé, le résultat attendu, les pièces jointes utiles (capture, journal) et deux classements indépendants que la leçon va distinguer : la gravité et la priorité. Sans étapes reproductibles, le développeur répond « impossible à reproduire » et l'anomalie meurt sans avoir été comprise.

La gravité mesure l'impact technique du défaut sur le système et ses utilisateurs : bloquante (une fonction essentielle est inutilisable ou des données sont perdues), majeure (fonction importante dégradée, sans contournement simple), mineure (gêne limitée, contournement facile), cosmétique (aspect visuel sans effet fonctionnel). La priorité, elle, mesure l'urgence de correction du point de vue du projet et de l'entreprise : elle peut donc diverger de la gravité. Un texte faux sur le reçu remis à chaque membre est de gravité mineure — le calcul est juste — mais de priorité haute, car il expose l'organisation à des litiges quotidiens ; inversement, un plantage rarissime sur une fonction d'archivage peu utilisée peut être de gravité majeure et de priorité moyenne.

Une campagne de test est l'exécution organisée, pendant une période donnée, des cas de test prévus par le plan, sur une version identifiée du produit. Elle se pilote avec des indicateurs simples : taux d'exécution (cas exécutés ÷ cas prévus), taux de réussite (cas réussis ÷ cas exécutés), nombre d'anomalies ouvertes par gravité. La décision finale — la recette — compare ces résultats aux critères de sortie fixés à l'avance dans le plan de test : par exemple 100 % des cas exécutés, au moins 95 % de réussite, zéro anomalie critique ouverte. Fixer ces seuils avant la campagne, et non après avoir vu les résultats, est ce qui distingue une démarche qualité d'une négociation.

L'essentiel
Une anomalie utile est reproductible : étapes numérotées, résultat observé contre résultat attendu, environnement précis. Gravité = impact technique ; priorité = urgence pour le projet — les deux axes sont indépendants. Une campagne se conclut par une recette comparée à des critères de sortie fixés avant l'exécution.

Exemple — Fiche d'anomalie complète : gravité mineure, priorité haute

```
FICHE D'ANOMALIE — BiblioIAM
Identifiant : ANO-2026-031 Statut : Ouverte
Projet / version : BiblioIAM 1.2.0 (recette)
Module : Emprunts — reçu de frais de retard
Environnement : Chrome 126, Windows 11,
serveur de recette, jeu de données n° 4
Rapporteur : A. Diallo (testeuse) Date : 12/03/2026

Titre : le reçu affiche les frais en centimes
au lieu de francs CFA.

Étapes de reproduction :
1. Se connecter avec un membre ayant 3 jours de retard.
2. Ouvrir « Mes emprunts » puis cliquer « Reçu ».
3. Lire le montant des frais de retard affiché.

Résultat observé : le reçu affiche « 30 000 » alors
que l'écran de détail affiche « 300 F ».
Résultat attendu : le reçu affiche « 300 F », comme
partout ailleurs dans l'application.

Gravité : MINEURE — le calcul interne est juste, seul
l'affichage du reçu est faux ; aucune donnée perdue.
Priorité : HAUTE — le reçu est remis à chaque membre :
risque de litiges au guichet et d'image dégradée.
Contournement : expliquer le montant oralement.
```

À retenir

- Pas d'étapes de reproduction, pas de correction : une anomalie non reproductible est une anomalie perdue.
- Gravité et priorité se classent séparément ; un défaut « petit » peut être urgent, et réciproquement.
- Les critères de sortie d'une campagne se fixent avant l'exécution, jamais après avoir vu les chiffres.

Exercice

Classe ces deux anomalies en gravité ET en priorité, en justifiant chaque classement : (a) l'application se ferme brutalement quand on ouvre le reçu d'un emprunt dont les frais sont de 0 F — ce cas ne concerne que les retours sans retard archivés, fonction utilisée une fois par mois par l'administration ; (b) le mot « Bibliothèque » est orthographié « Bibliotèque » dans le titre de la page d'accueil publique.

✔ Corrigé
(a) Gravité majeure : un plantage est une défaillance franche de l'application, même si le périmètre est étroit ; selon la grille de l'équipe, « critique/bloquante » serait réservé à une fonction essentielle inutilisable, ce qui n'est pas le cas ici. Priorité moyenne : la fonction touchée est mensuelle et administrative, un contournement existe (ne pas ouvrir ces reçus), la correction peut attendre le prochain cycle sans exposer les membres. (b) Gravité cosmétique (la plus faible) : aucun effet fonctionnel, aucune donnée touchée. Priorité haute : la page d'accueil est la vitrine publique de l'institution, vue par tous, et la correction prend quelques minutes — coût dérisoire, bénéfice d'image immédiat. Les deux cas prouvent l'indépendance des axes : classer « gravité = priorité » par réflexe aurait retardé (b) et précipité (a) à contresens.

Leçon 7 — Projet : plan de test complet d'une application réelle

Tout ce que ce cours a construit pièce par pièce se rassemble dans un document : le plan de test. Hérité des pratiques décrites par l'ISTQB et les standards de type IEEE 829, il répond, pour une version donnée d'une application réelle, aux questions : quoi tester, à quels niveaux, avec quelles techniques, dans quel environnement, selon quel planning, et avec quels critères pour décider d'arrêter et de livrer. Un plan de test n'est pas un formulaire bureaucratique : c'est le contrat qui évite, le jour de la livraison, l'affrontement stérile entre « il reste des bogues » et « il faut sortir ».

Un plan complet comprend typiquement : l'objet et le périmètre (inclus et explicitement exclus), les types et niveaux de test retenus en cohérence avec la pyramide, l'approche de conception des cas (classes d'équivalence, valeurs limites, parcours critiques), les environnements et les jeux de données, les critères d'entrée (ce qui doit être prêt avant de commencer) et de sortie (les seuils de la recette), les livrables (cas de test, fiches d'anomalie, bilan de campagne), les rôles et le planning. L'extrait ci-dessous, pour la version 1.3 de BiblioIAM, montre le niveau de précision attendu : chaque phrase doit pouvoir être vérifiée objectivement par quelqu'un d'extérieur à l'équipe.

Le TP qui suit te fait produire ce plan, puis l'exécuter en miniature sur un module réel volontairement fautif : tu y retrouveras les tests unitaires de la leçon 3, la couverture de la leçon 5 et la fiche d'anomalie de la leçon 6, réunis dans le geste professionnel complet — concevoir, exécuter, mesurer, documenter, décider. C'est exactement la boucle qu'un pipeline d'intégration continue automatise ensuite à chaque modification : le plan de test est la version humaine et argumentée de ce que la machine répétera sans fatigue.

L'essentiel
Le plan de test est le document de synthèse du cours : périmètre, niveaux, techniques, environnements, planning, livrables et surtout critères d'entrée et de sortie vérifiables. Il transforme la qualité d'une opinion en décision traçable — et le TP le met en œuvre de bout en bout.

Exemple — Plan de test de BiblioIAM v1.3 (extrait structuré)

```
PLAN DE TEST — BiblioIAM v1.3 (extrait)
1. Objet : vérifier la version 1.3 (emprunts, retours,
frais de retard, recherche au catalogue).
2. Périmètre inclus : modules Emprunts et Catalogue,
API publique, rôles membre et bibliothécaire.
Exclu : le module Statistiques (reporté en v1.4).
3. Niveaux et types : unitaires (développeurs),
API et intégration, bout en bout sur 6 parcours
critiques, tests exploratoires de 2 h par module.
4. Critères d'entrée : environnement de recette
disponible, jeux de données n° 4 chargés, aucune
anomalie bloquante de la v1.2 encore ouverte.
5. Critères de sortie : 100 % des cas exécutés,
au moins 95 % de réussite, zéro anomalie critique
ou majeure ouverte, couverture du module Emprunts
d'au moins 80 %.
6. Livrables : cas de test, fiches d'anomalie,
bilan de campagne signé par le chef de projet.
7. Planning : conception j1 à j3, exécution j4 à j8,
corrections et retests j9 à j10, bilan j11.
```

À retenir

- Un plan de test précis est vérifiable par un tiers : chaque critère est chiffré ou observable, jamais vague.
- Le périmètre explicite ce qui est exclu : c'est là que naissent les malentendus de livraison.
- Plan, exécution, bilan : la campagne réelle du TP applique ce document à un module fautif.

Exercice

Pour une application de relevé de notes d'un institut, rédige les critères d'entrée et les critères de sortie d'une campagne de test de la fonction « calcul des moyennes ». Exige au moins deux critères d'entrée et trois critères de sortie, tous vérifiables objectivement (chiffrés ou binaires), et explique en une phrase pourquoi interdire « la campagne commence quand le développeur se sent prêt » comme critère d'entrée.

✓ Corrigé

Critères d'entrée possibles : la version à tester est figée et installée sur l'environnement de recette (binaire : oui/non) ; le jeu de données de référence — bulletins calculés à la main — est validé par un enseignant (oui/non) ; aucune anomalie bloquante connue sur la fonction moyennes. Critères de sortie possibles : 100 % des cas prévus exécutés ; au moins 98 % de réussite, car un calcul de moyenne faux n'est pas tolérable longtemps ; zéro anomalie critique ou majeure ouverte sur le calcul (les écarts d'arrondi mineurs doivent être documentés et acceptés par écrit) ; couverture du module de calcul d'au moins 90 %. « Quand le développeur se sent prêt » n'est pas un critère : il n'est ni observable ni contestable, donc impossible à opposer le jour où la campagne démarre sur une version inachevée — un critère doit pouvoir être vérifié par quelqu'un d'autre que la personne qu'il engage.

Travaux dirigés (TD)

TD 1 — Concevoir les cas de test d'un tarif par âges

La bibliothèque applique un tarif d'inscription selon l'âge : moins de 12 ans → inscription refusée (la fonction renvoie -1) ; de 12 à 17 ans → 1 000 F ; de 18 à 25 ans → 2 500 F ; de 26 à 60 ans → 4 000 F ; plus de 60 ans → 2 000 F. La fonction tarif_inscription(age) reçoit un âge entier déjà validé (0 à 120). 1) Liste les classes d'équivalence. 2) Donne l'ensemble minimal des valeurs limites à tester, avec le résultat attendu pour chacune. 3) Explique pourquoi un seul représentant par classe ne suffirait pas pour cette fonction.

✔ Corrigé du TD 1

1) Six classes d'équivalence : [moins de 12] refus ; [12-17] 1 000 F ; [18-25] 2 500 F ; [26-60] 4 000 F ; [plus de 60] 2 000 F ; soit cinq classes valides de traitement différent, plus la classe « refus » — chaque classe correspond à une branche de décision. 2) Valeurs limites (vérifiées par exécution de la fonction) : 11 → -1 (dernier âge refusé) ; 12 → 1 000 (premier âge accepté) ; 17 → 1 000 (fin du tarif jeune) ; 18 → 2 500 (début du tarif étudiant) ; 25 → 2 500 (fin) ; 26 → 4 000 (début du tarif adulte) ; 60 → 4 000 (fin) ; 61 → 2 000 (début du tarif senior). Huit cas suffisent à verrouiller les quatre frontières. 3) Un représentant par classe (par exemple 15, 20, 40, 70) ne détecterait pas une erreur de borne du type « age < 17 » au lieu de « age <= 17 », qui ne se manifeste qu'à 17 et 18 ans exactement : les classes disent QUOI traiter, les limites disent OÙ les traitements se séparent — c'est aux frontières que vivent les défauts.

TD 2 — Lire un bilan de campagne et décider la recette

Campagne de recette de BiblioIAM v1.3. Plan de test : 240 cas prévus. Résultats : 228 cas exécutés, dont 205 réussis, 15 en échec et 8 bloqués. Les échecs ont produit 12 anomalies produit (2 critiques, 4 majeures, 6 mineures) ; les 3 autres échecs viennent d'un environnement défaillant (aucune anomalie produit). Les critères de sortie du plan étaient : au moins 98 % des cas exécutés ; au moins 95 % de réussite sur les cas exécutés ; zéro anomalie critique ouverte. 1) Calcule le taux d'exécution et le taux de réussite. 2) Compare chaque critère à son seuil. 3) Rédige la décision de recette en trois phrases maximum, avec la conduite à tenir.

✔ Corrigé du TD 2

1) Taux d'exécution : 228 ÷ 240 = 95,0 %. Taux de réussite : 205 ÷ 228 = 89,9 % (contrôle : 205 + 15 + 8 = 228 cas exécutés, le compte est complet). 2) Les trois critères échouent : exécution 95,0 % < 98 % ; réussite 89,9 % < 95 % ; et 2 anomalies critiques sont ouvertes alors que le seuil exige zéro. 3) Décision : la recette est REFUSÉE. Conduite à tenir : corriger d'abord les 2 anomalies critiques et les 4 majeures, réparer l'environnement pour débloquer les 8 cas restants, puis ré-exécuter les cas en échec et bloqués (retests) ainsi que les tests de non-régression des zones corrigées, avant une nouvelle décision. Ce refus n'est pas un échec du projet : c'est le mécanisme prévu par le plan, et il vaut mieux un refus documenté en recette qu'une panne documentée chez les utilisateurs.

Travaux pratiques (TP)

TP 1 — Tester, mesurer et corriger un module fautif

On te confie le module emprunts.py ci-dessous, accompagné de son règlement : frais_retard applique 100 F par jour pendant la première semaine (7 jours), puis 200 F par jour au-delà, avec un plafond de 3 000 F ; un nombre de jours négatif lève ValueError ; peut_emprunter refuse tout membre bloqué, et refuse à partir de 3 emprunts en cours. def frais_retard(jours): if jours < 0: raise ValueError("jours négatif impossible") if jours == 0: return 0 if jours <= 5: return 100 * jours return min(500 + 200 * (jours - 5), 3000) def peut_emprunter(nb_emprunts, bloque): if bloque: return False return nb_emprunts < 3 Travail demandé : 1) Écris une suite pytest complète (classes d'équivalence et valeurs limites pour les deux fonctions, cas d'erreur inclus). 2) Exécute-la et relève précisément les échecs. 3) Pour l'écart constaté, rédige une fiche d'anomalie complète (gravité et priorité justifiées). 4) Mesure la couverture du module avec coverage.py. 5) Corrige le module (sans toucher aux tests) jusqu'à obtenir une suite entièrement verte, puis remesure la couverture.

✔ Attendus du TP 1

Attendus (vérifiés par exécution avec pytest 9.1.1 et coverage.py). 1-2) La suite de référence compte 9 cas : 7 pour frais_retard (0, 1, 5, 7, 10, 30 jours et le cas négatif) et 2 pour peut_emprunter. Résultat sur le module fourni : 2 failed, 7 passed. Les échecs sont exactement les cas qui franchissent la première semaine : frais_retard(7) renvoie 900 au lieu de 700, et frais_retard(10) renvoie 1 500 au lieu de 1 300. 3) Fiche d'anomalie attendue : le seuil de majoration est codé à 5 jours (base 500) au lieu de 7 jours (base 700) : tous les frais des retards de 6 jours et plus sont faux, toujours en défaveur du membre. Gravité majeure (calcul financier faux, pas de contournement), priorité haute (le guichet encaisse ces frais chaque jour). 4) Couverture du module fourni : la suite exécute toutes les lignes, l'écart est un défaut de VALEUR, pas de couverture — démonstration concrète que 100 % de couverture ne suffit pas. 5) Correction : remplacer le seuil 5 par 7 et la base 500 par 700, soit return min(700 + 200 * (jours - 7), 3000) au-delà de 7 jours. Après correction, toujours sans toucher aux tests : 9 passed, et la couverture du module corrigé est de 100 % (12 instructions, 0 manquée). Le livrable final rassemble : la suite, la fiche, les deux rapports de couverture et le module corrigé.

Exercices corrigés supplémentaires

Exercice 1

Un atelier de formation accueille au maximum 30 inscrits. La fonction peut_inscrire(nb_inscrits) renvoie True si une place reste libre, False sinon. Donne les valeurs limites à tester et le résultat attendu pour chacune, puis explique pourquoi tester 0 et 1 reste utile même si la frontière est à 30.

✔ Corrigé
La frontière est entre 29 et 30 inscrits : peut_inscrire(29) vaut True (la 30e et dernière place) et peut_inscrire(30) vaut False (atelier complet). On ajoute les voisines de robustesse : 28 (True, intérieur proche) et 31 (False, au-delà, par exemple si un compteur incohérent arrive jusqu'à la fonction). Tester 0 (True : atelier vide) et 1 (True) reste utile : ce sont les cas nominaux qui vérifient que la fonction ne refuse pas tout par principe — un test qui n'a jamais vu passer un True ne prouve pas que la fonction sait accepter. Un jeu complet et économe : 0, 1, 29, 30, 31, soit cinq cas pour verrouiller le comportement entier d'une fonction à seuil.

Exercice 2

Voici une fiche d'anomalie telle qu'un utilisateur l'a déposée : « Titre : ça marche pas. Description : Quand je clique sur le bouton pour emprunter, des fois ça ne marche pas. C'est urgent, merci de corriger au plus vite. » Liste tout ce qui manque pour que cette fiche soit exploitable, puis réécris-la correctement en inventant les précisions raisonnables (environnement, étapes, résultats) pour le cas suivant : sur BiblioIAM, le bouton « Emprunter » reste sans effet quand le membre a déjà 3 emprunts en cours, alors qu'un message de refus devrait s'afficher.

✔ Corrigé
Manques de la fiche d'origine : aucun identifiant ni version ; titre inexploitable ; aucune étape de reproduction (« des fois » n'est pas une condition) ; pas d'environnement (navigateur, compte utilisé) ; pas de résultat attendu — on ne sait même pas quel comportement serait correct ; pas de pièces jointes ; et « urgent » confond priorité souhaitée et gravité, sans aucun élément pour les évaluer. Version corrigée : Titre : « Aucun retour visuel quand l'emprunt est refusé (limite de 3 atteinte) ». Version : BiblioIAM 1.2.0, Chrome 126, compte membre n° 12 du jeu de données n° 4 (3 emprunts en cours). Étapes : 1) se connecter avec ce membre ; 2) ouvrir un livre disponible ; 3) cliquer « Emprunter ». Observé : rien ne se passe, aucun message. Attendu : affichage du message « Limite de 3 emprunts atteinte » et aucun emprunt créé. Gravité majeure (l'utilisateur ne peut ni réussir ni comprendre l'échec), priorité haute (parcours principal de l'application).

Exercice 3

Un rapport de couverture donne : fichier a.py, 40 instructions, 40 exécutées ; fichier b.py, 55 instructions, 41 exécutées. 1) Calcule la couverture de chaque fichier puis la couverture totale (ce n'est pas la moyenne des deux pourcentages). 2) Le chef de projet conclut : « à 85 %, on peut livrer, il ne reste presque aucun bogue ». Corrige cette conclusion en deux arguments, dont un appuyé sur un petit exemple de test qui couvre sans vérifier.

✔ Corrigé
1) a.py : 40 ÷ 40 = 100 %. b.py : 41 ÷ 55 = 74,5 %. Total : (40 + 41) ÷ (40 + 55) = 81 ÷ 95 = 85,3 % — on additionne les instructions avant de diviser, car les fichiers n'ont pas la même taille ; la moyenne naïve (100 + 74,5) ÷ 2 = 87,25 % serait fausse. 2) La conclusion est doublement fautive. D'abord, la couverture mesure des lignes exécutées, pas des comportements vérifiés : le test « def test_retard(): frais_retard(10) » sans aucun assert exécute la fonction, couvre ses lignes, et laisserait passer un résultat de 1 500 au lieu de 1 300 — c'est exactement le défaut du TP de ce cours, présent dans un module pourtant couvert à 100 %. Ensuite, le pourcentage ne dit pas OÙ sont les 15 % manquants : s'ils concentrent le calcul des frais ou la sécurité, le risque restant est énorme ; s'ils ne touchent que des messages d'affichage, il est faible. Décider une livraison exige la colonne des lignes manquantes et des assertions sérieuses, pas un seul chiffre.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. L'assurance qualité se distingue du contrôle qualité parce qu'elle…

   A) teste le produit fini avant livraison

   B) organise les processus pour prévenir les défauts

   C) rédige les fiches d'anomalie à la place des testeurs

2. Un test unitaire vérifie…

   A) une fonction ou une méthode, de façon isolée

   B) le parcours complet d'un utilisateur

   C) la tenue du serveur sous charge

3. Une couverture de code de 100 % signifie que…

   A) le logiciel ne contient aucun bogue

   B) le produit peut être livré sans recette

   C) chaque ligne a été exécutée au moins une fois pendant les tests

4. Une anomalie d'affichage visible par tous les membres, sans aucun impact sur les calculs, est typiquement…

   A) de gravité forte et de priorité faible

   B) de gravité faible, avec une priorité pouvant être élevée

   C) à classer gravité = priorité dans tous les cas

5. Les tests de non-régression servent à vérifier que…

   A) les nouveautés fonctionnent

   B) la couverture augmente

   C) une correction ou une évolution n'a rien cassé d'existant

6. Les critères de sortie d'une campagne de test doivent être…

A) fixés avant l'exécution, à partir du plan de test

B) négociés après avoir vu les résultats

C) identiques pour tous les projets

7. En conception boîte noire, les cas de test sont construits à partir…

A) des spécifications et des comportements attendus, sans regarder le code

B) de la liste des lignes de code à couvrir

C) des seuls cas d'erreur connus

8. Un test dit « instable » (flaky) est un test qui…

A) échoue à tous les coups

B) teste les performances

C) passe ou échoue selon les exécutions, sans changement du code

9. Dans un pipeline d'intégration continue, la suite de tests unitaires doit être lancée…

A) à chaque intégration de code

B) une fois par mois

C) uniquement la veille de la livraison

10. Le contenu indispensable d'une fiche d'anomalie est…

A) le nom du développeur responsable du défaut

B) des étapes de reproduction précises, le résultat observé et le résultat attendu

C) une date de correction imposée au développeur

Corrigés du QCM

1. Réponse B — L'assurance qualité est orientée processus et prévention (normes, revues, listes de contrôle) ; le contrôle qualité et le test vérifient le produit obtenu.

2. Réponse A — Le test unitaire cible la plus petite unité testable, isolée de ses dépendances, avec un retour rapide et précis.

3. Réponse C — La couverture mesure l'exécution, pas la vérification : elle ne dit ni si les assert sont bons, ni si les règles métier sont justes.

4. Réponse B — Gravité (impact technique) et priorité (urgence pour le projet) sont deux axes indépendants : un défaut cosmétique très visible peut être urgent.

5. Réponse C — Après chaque changement, on rejoue les tests des comportements déjà validés pour détecter les effets de bord.

6. Réponse B — Des seuils définis à l'avance (taux d'exécution, réussite, anomalies critiques ouvertes) rendent la décision de recette objective et contestable.

7. Réponse A — Classes d'équivalence et valeurs limites sont des techniques boîte noire : on raisonne sur les entrées et les sorties attendues.

8. Réponse C — Dépendance à l'heure, au réseau ou à l'ordre d'exécution : un test instable détruit la confiance dans toute la suite et doit être corrigé ou isolé.

9. Réponse B — Des tests rapides à chaque commit donnent un retour immédiat, quand la correction coûte le moins cher.

10. Réponse B — Sans reproduction fiable ni écart observé/attendu explicite, l'anomalie n'est ni corrigeable ni vérifiable après correction.

Résumé — les notions importantes

• La qualité logicielle se mesure (modèle ISO/IEC 25010) ; l'assurance qualité prévient par les processus, le contrôle qualité inspecte, le test exécute — et le test prouve la présence de défauts, jamais leur absence.

• La pyramide des tests répartit l'effort : beaucoup de tests unitaires rapides à la base, des tests d'API et d'intégration au milieu, quelques tests de bout en bout au sommet ; l'inversion de cette pyramide est l'anti-modèle du « cornet de glace ».

• Avec pytest, on construit les cas par classes d'équivalence et valeurs limites, on teste les erreurs attendues, et l'on obtient des suites relançables à chaque commit (7 tests de frais_retard, tous verts, dans ce cours).

• Les tests d'API vérifient les contrats (statuts HTTP et contenus) contre un environnement de test maîtrisé ; les tests de bout en bout couvrent uniquement les parcours critiques.

• La couverture de code se lit par ses lignes manquantes (biblio.py : 58 %, deux fonctions sans aucun test) et s'utilise comme seuil plancher dans le pipeline — jamais comme preuve d'absence de bogues.

• Gravité et priorité sont deux axes indépendants ; une anomalie professionnelle est reproductible, avec résultat observé contre résultat attendu.

• Une campagne de test se pilote par des indicateurs (exécution, réussite, anomalies par gravité) et se conclut par une recette comparée à des critères de sortie fixés à l'avance.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Concepts de qualité et modèles d'assurance qualité
La qualité couvre bien plus que l'absence de bogues : utilisabilité, fiabilité, sécurité, maintenabilité… ; Assurance qualité = prévenir par les processus ; contrôle qualité = inspecter ; test = exécuter et observer. ; Comme on ne peut pas tout tester, on priorise par le risque : critique, souvent modifié, sensible.

Fiche 2 — Stratégies de test : la pyramide des tests
Plus un test est haut dans la pyramide, plus il est lent, coûteux à maintenir et imprécis sur la cause d'échec. ; Automatiser au plus bas niveau capable de vérifier le comportement visé. ; Une stratégie dit aussi ce qui ne sera pas automatisé, et à quel moment chaque étage est exécuté.

Fiche 3 — Tests unitaires automatisés avec un framework de test
Un test unitaire est isolé, déterministe et rapide ; s'il dépend du réseau ou de l'horloge, ce n'est plus un test unitaire. ; Classes d'équivalence pour couvrir les comportements, valeurs limites pour traquer les erreurs de comparaison. ; Tester aussi les cas d'erreur : une exception prévue et vérifiée vaut mieux qu'un plantage découvert en production.

Fiche 4 — Tests d'API et tests de bout en bout
Un test d'API vérifie le contrat : code de statut, structure et contenu — pas seulement « ça répond ». ; Environnement de test maîtrisé (fixture, données connues) = tests reproductibles, condition de leur crédibilité. ; Les E2E couvrent peu de parcours, mais les plus critiques ; leur lenteur est un coût assumé, pas un défaut à nier.

Fiche 5 — Couverture de code et intégration aux pipelines
Lire un rapport de couverture : d'abord les lignes manquantes, ensuite seulement le pourcentage. ; Couverture = exécution mesurée, pas correction prouvée ; un test sans assert couvre sans vérifier. ; Dans un pipeline : un seuil plancher qui bloque la fusion, pas une course au 100 %.

Fiche 6 — Gestion des anomalies et campagnes de test
Pas d'étapes de reproduction, pas de correction : une anomalie non reproductible est une anomalie perdue. ; Gravité et priorité se classent séparément ; un défaut « petit » peut être urgent, et réciproquement. ; Les critères de sortie d'une campagne se fixent avant l'exécution, jamais après avoir vu les chiffres.

Fiche 7 — Projet : plan de test complet d'une application réelle
Un plan de test précis est vérifiable par un tiers : chaque critère est chiffré ou observable, jamais vague. ; Le périmètre explicite ce qui est exclu : c'est là que naissent les malentendus de livraison. ; Plan, exécution, bilan : la campagne réelle du TP applique ce document à un module fautif.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — [FR] ISTQB Foundation v4 — QCM & Examens Blancs : 2026-Questions (avec explications)

Chaîne : QualityLab Academy · Langue : Français · Niveau : Intermédiaire
Un entraînement complet aux questions de la certification ISTQB Foundation v4, avec des explications détaillées à chaque réponse. Idéal pour consolider le vocabulaire et les principes de cette première leçon (et réviser avant un examen).

https://www.youtube.com/watch?v=rUDMAWkSjic

Leçon 1 — 2.1. Tester dans le contexte d'un cycle de vie du développement logiciel

Chaîne : Mariem Mili · Langue : Français · Niveau : Intermédiaire

Comment les activités de test s'articulent avec le cycle de vie du développement (en V, itératif, DevOps) et pourquoi tester tôt — le fameux shift left. Un bon complément sur la place du test dans un projet, avant même les outils.

https://www.youtube.com/watch?v=tVOzX2teJwY

Leçon 2 — Construire une Stratégie de Test Efficace

Chaîne : La Guilde ! - Expertise en Quality Engineering · Langue : Français · Niveau : Intermédiaire

De la définition d'une stratégie de test à sa mise en œuvre : niveaux de test, pyramide, environnements, indicateurs et automatisation. La vidéo à regarder pour relier la pyramide de cette leçon aux décisions concrètes d'une équipe.

https://www.youtube.com/watch?v=hdol55mYGZ8

Leçon 3 — 15.09 | Écrire des tests unitaires et d’intégration pour un ERP Moderne avec Spring Boot | LIVE #26

Chaîne : Ismael Loko · Langue : Français · Niveau : Intermédiaire

Une session de codage en direct où des tests unitaires et d'intégration sont écrits sur une vraie application métier (un ERP, en Java/Spring). Le framework diffère de pytest, mais la démarche — quoi tester, comment isoler, comment structurer — est exactement celle de la leçon.

https://www.youtube.com/watch?v=sas3Yk4Mrko

Leçon 6 — Test logiciel : COMMENT CONCEVOIR LES CAS DE TEST

Chaîne : qualite logiciel · Langue : Français · Niveau : Débutant à intermédiaire

Les techniques de conception des cas de test (classes d'équivalence, valeurs limites, tables de décision) expliquées pas à pas dans la tradition ISTQB. Parfait pour préparer le TD et structurer ensuite des campagnes de test complètes.

https://www.youtube.com/watch?v=NZDvPYPBMEE

Sources et références

• Syllabus ISTQB Certified Tester Foundation Level (CTFL) et glossaire officiel du CFTL (Comité Français des Tests Logiciels) — la référence des concepts : niveaux de test, techniques de conception, gestion des anomalies.

• Documentation officielle de pytest (docs.pytest.org) — découverte des tests, fixtures, pytest.raises et bonnes pratiques d'organisation des suites.

• Documentation Python : module unittest (docs.python.org/fr/3/library/unittest.html) — le framework de test de la bibliothèque standard, et ses concepts de cas de test et de suites.

• Documentation de coverage.py (coverage.readthedocs.io) — mesure de la couverture, rapports, couverture de branches et intégration dans les pipelines.

• Norme ISO/IEC 25010 (modèle de qualité des systèmes et logiciels) et Mike Cohn, « Succeeding with Agile » (2009), pour la pyramide des tests — à consulter via la bibliothèque de l'IAM.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !